



OKX Smart Wallet 2.0.0

Security Audit Report

Prepared by

OKX Web3 Audit Team

24 September 2026

Contents

1	Overview	3
1.1	Project Introduction	3
1.2	Audit Summary	3
1.3	Audit Scope	3
2	Audit Summary	4
2.1	Audit Methodology	4
2.2	Audit Process	4
2.3	Risk Classification and Description	5
2.4	Results	5
3	Vulnerabilities	6
3.1	Low - Any Owner Can Cancel Another Owner's Transfer Authorization	6
3.2	Low - TWA Signatures Do Not Bind the Implementation	8
3.3	Low - Mixed Chainless Operations Can Leave a Stale Queue Valid	9
4	Disclaimer	10
5	About OKX Web3 Audit Team	10

1. Overview

1.1 Project Introduction

AA Auth is a Solidity smart-wallet authorization module that supports transfer authorizations, account ownership management, signature validation, hook-based settlement, and chain-independent operations. This audit reviewed the specified smart-wallet contracts and libraries for authorization integrity, nonce isolation, upgrade-domain separation, and consistency across execution paths.

1.2 Audit Summary

Ecosystem	EVM
Language	Solidity
Repository	https://gitlab.okg.com/web3-wallet/smart-wallet-infra/okx-smart-wallet-dev
Base Commit	57bd1b39f87a247e6484b65eaa9ebc5a872ebe37
Final Commit	66525173b6a0307afc949cac3ef371d5e4ab1539

1.3 Audit Scope

```
src/
├── TransferWithAuthorization.sol
├── SmartWallet.sol
├── NonceManager.sol
├── OwnerManager.sol
├── ExecutionManager.sol
└── ValidationManager.sol

src/interfaces/
├── ITransferWithAuthorization.sol
├── IHookTransferAuthorization.sol
├── IHook.sol
├── INonceManager.sol
└── IOwnerManager.sol

src/libraries/
├── HookLib.sol
├── ChainlessLib.sol
├── Static.sol
├── CallLib.sol
├── DecodeLib.sol
├── ECDSAValidatorLib.sol
├── MerkleProofProcessor.sol
└── PasskeyValidatorLib.sol
```

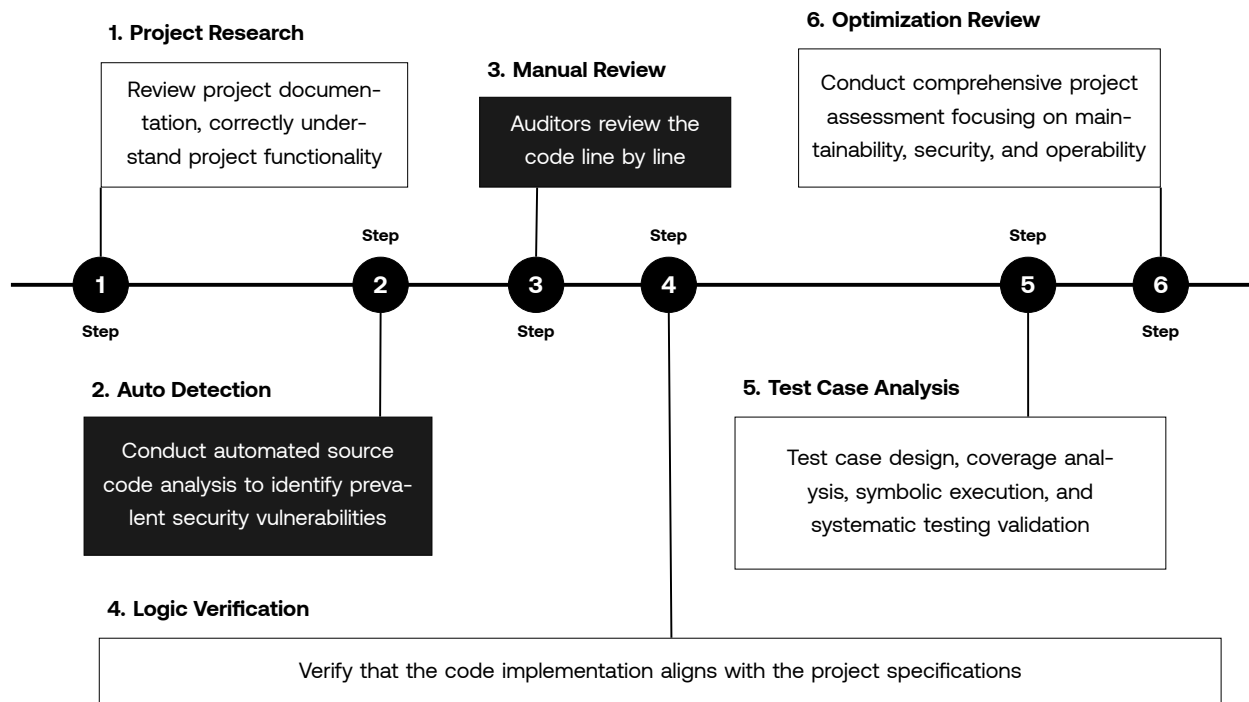
2. Audit Summary

2.1 Audit Methodology

The audit team conducted comprehensive analysis of the contract code through deep understanding of the project's design purpose, operating principles, and implementation methods. By mapping function call relationships, potential security vulnerabilities were systematically identified, with detailed problem descriptions and corresponding remediation recommendations provided.

2.2 Audit Process

The smart contract security audit follows a 6-phase process: Project Research, Automated Detection, Manual Review, Logic Verification, Test Case Analysis, and Optimization Review. During manual auditing, auditors perform comprehensive code review to identify vulnerabilities and provide detailed solutions. After completing all phases, the lead auditor communicates findings with the project team. Following the team's responses, we deliver final audit reports to the project team.



2.3 Risk Classification and Description

Risk items are classified into 5 levels: Critical, High, Medium, Low, and Informational. Critical risks require immediate resolution and re-audit before final report delivery; unresolved critical risks result in audit failure.

Risk Level	Risk Description
Critical	Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
High	High risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.
Medium	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
Low	Low risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
Informational	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

2.4 Results

The audit results are divided into two parts: one part is the vulnerability summary of the project audit, and the other part is the detailed vulnerability list.

Vulnerability Summary

Critical	High	Medium	Low	Informational	Total
0	0	0	3	0	3

Vulnerability list

No.	Severity	Vulnerability	Category	Status
1	Low	Any Owner Can Cancel Another Owner's Transfer Authorization	Business Logic	Fixed
2	Low	TWA Signatures Do Not Bind the Implementation	Business Logic	Fixed
3	Low	Mixed Chainless Operations Can Leave a Stale Queue Valid	Business Logic	Confirmed

Status Definitions

- **Open:** The audit team has notified the project team of the vulnerability, but no reasonable remediation has been implemented.
- **Fixed:** The project team has addressed the vulnerability and the fix has been verified by the audit team.
- **Confirmed:** The project team has confirmed awareness of the vulnerability risk but considers it controllable.

3. Vulnerabilities

This section outlines the risk items identified through manual review and auditing tools. Each item includes the specific file path and code location, along with the assigned risk level.

3.1 Low - Any Owner Can Cancel Another Owner's Transfer Authorization

Location	File	Category	Status	Severity
L101	src/TransferWithAuthorization.sol	Business Logic	Fixed	Low

Description

The signature for `cancelTransferAuthorization` binds only the `authorizationNonce`; it does not bind the owner who created the authorization. Because authorization nonces are shared at the wallet level, any registered owner can sign a cancellation for an unexecuted authorization created by another owner, preventing the original transaction from being executed.

Related Code

```
101 function cancelTransferAuthorization(  
102     bytes32 authorizationNonce,  
103     bytes calldata signature  
104 ) external {  
105     TransferAuthorizationStorage storage $ =  
106         _getTransferAuthorizationStorage();  
107     if ($.authorizationStates[authorizationNonce]) {  
108         revert AuthorizationAlreadyUsed(authorizationNonce);  
109     }  
110  
111     if (signature.length == 0) {  
112         if (msg.sender != address(this)) revert NotFromSelf();  
113     } else {  
114         bytes32 structHash = keccak256(  
115             abi.encode(  
116                 CANCEL_TRANSFER_AUTHORIZATION_TYPEHASH,  
117                 authorizationNonce  
118             )  
119         );  
120         _verifyTwaSignature(structHash, signature);  
121     }  
122  
123     $.authorizationStates[authorizationNonce] = true;  
124     emit TransferAuthorizationCanceled(authorizationNonce);  
125 }
```

Recommendation

Bind the original authorizing owner to the cancellation message and restrict cancellation to that owner or an explicitly authorized administrator.

Project Team Feedback

Team Response	Accepted. The nonce was changed to a two-dimensional mapping (key hash to nonce to Boolean), and cancellation is restricted to the administrator or the original authorizing owner.
Re-audit Result	The remediation was verified.

3.2 Low - TWA Signatures Do Not Bind the Implementation

Location	File	Category	Status	Severity
L157	TransferWithAuthorization.sol	Business Logic	Fixed	Low

Description

The EntryPoint and Relayer execution paths bind IMPLEMENTATION into their signature hashes, associating every authorization with a specific wallet implementation. After an upgrade, signatures created for the previous implementation therefore cannot be reused through those paths. The TWA signature hash did not bind IMPLEMENTATION, creating inconsistent upgrade-domain separation.

Related Code

```

157 bytes32 structHash = keccak256(
158     abi.encode(
159         typeHash,
160         token,
161         address(this),
162         to,
163         value,
164         validAfter,
165         validBefore,
166         authorizationNonce
167     )
168 );
169 (bytes32 keyHash, uint256 settings) =
170     _verifyTwaSignature(structHash, signature);
171
172 $.authorizationStates[authorizationNonce] = true;
173 _settleWithHook(keyHash, settings, token, to, value);

```

Recommendation

Align the TWA path with the EntryPoint and Relayer paths by adding IMPLEMENTATION to the signed TWA data structure.

Project Team Feedback

Team Response	Accepted. The implementation address was added to the signed data.
Re-audit Result	The remediation was verified.

3.3 Low - Mixed Chainless Operations Can Leave a Stale Queue Valid

Location	File	Category	Status	Severity
L34	src/libraries/ChainlessLib.sol	Business Logic	Confirmed	Low

Description

addOwner and upgradeToAndCall should use distinct operation types and nonces, but the implementation allows both operation types in one batch. Because the whole batch carries only one nonce, execution advances only one operation queue. A previously signed transaction in the other queue can remain valid and may later execute an outdated owner addition or contract upgrade.

Related Code

```

34  function canSkipChainIdValidation(
35      bytes4 functionSelector
36  ) internal pure returns (bool) {
37      if (
38          functionSelector == OwnerManager.addOwner.selector ||
39          functionSelector == UUPSUpgradeable.upgradeToAndCall.selector
40      ) {
41          return true;
42      }
43      return false;
44  }

```

Recommendation

Reject chainless batches that combine operations with different operation types, ensuring that each queue advances independently and invalidates stale signatures as intended.

Project Team Feedback

Team Response	The recommendation was acknowledged, but no code change was made. Mixed operations remain intentionally permitted.
Re-audit Result	The project team confirmed awareness and acceptance of the risk.

4. Disclaimer

This audit report only covers the specific audit types stated herein. We assume no responsibility for unknown security vulnerabilities outside this scope.

We rely on audit reports issued before existing attacks or vulnerabilities are published. For future or new vulnerabilities, we cannot guarantee project security impact and assume no responsibility.

Our security audit analysis should be based on documents provided by the project before report release (including contract code). These materials should not contain false information, tampering, deletion, or concealment. If provided materials are false, inaccurate, missing, tampered, deleted, concealed, or modified after report release, we assume no responsibility for resulting losses and adverse effects.

The project team should understand our audit report is based on provided materials and current technical capabilities. Due to institutional technical limitations, our report may not detect all risks. We encourage continued testing and auditing by the development team and stakeholders.

The project team must ensure compliance with applicable laws and regulations.

The audit report is for reference only. Its content, acquisition methods, usage, and related services cannot serve as basis for investment, taxation, legal, regulatory, or construction decisions. Without our prior written consent, the project team may not reference, cite, display, or distribute report content to third parties. Any resulting losses shall be borne by the project team.

This report does not cover contract compiler bugs or scope beyond programming languages. Smart contract risks from underlying vulnerabilities should be borne by the project team.

Force majeure includes unforeseeable, unavoidable events like wars, natural disasters, strikes, epidemics, and legal/regulatory changes preventing contract performance. When occurring, neither party breaches contract obligations. For unaffected economic responsibilities, the project team should pay for completed work.

5. About OKX Web3 Audit Team

OKX Web3 Audit Team specializes in blockchain security with expertise in smart contract auditing, token security assessment, and Web3 security tool development. We provide comprehensive security solutions for OKX's internal Web3 projects, conduct pre-listing token audits, and develop security tools to protect OKX Web3 wallet users. Our team combines automated analysis with manual review to deliver thorough security assessments and maintain the highest security standards in the Web3 ecosystem.